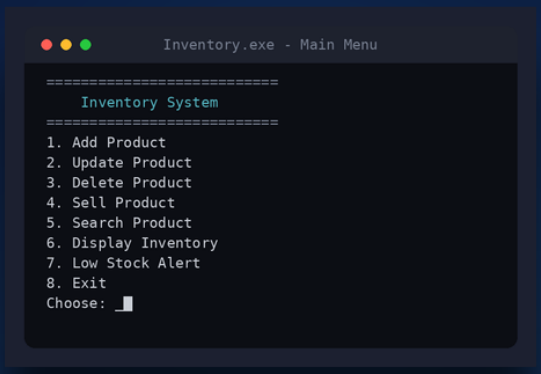
Cover - Inventory Management System (C++)

C++ · ALGORITHMS · DATA STRUCTURES

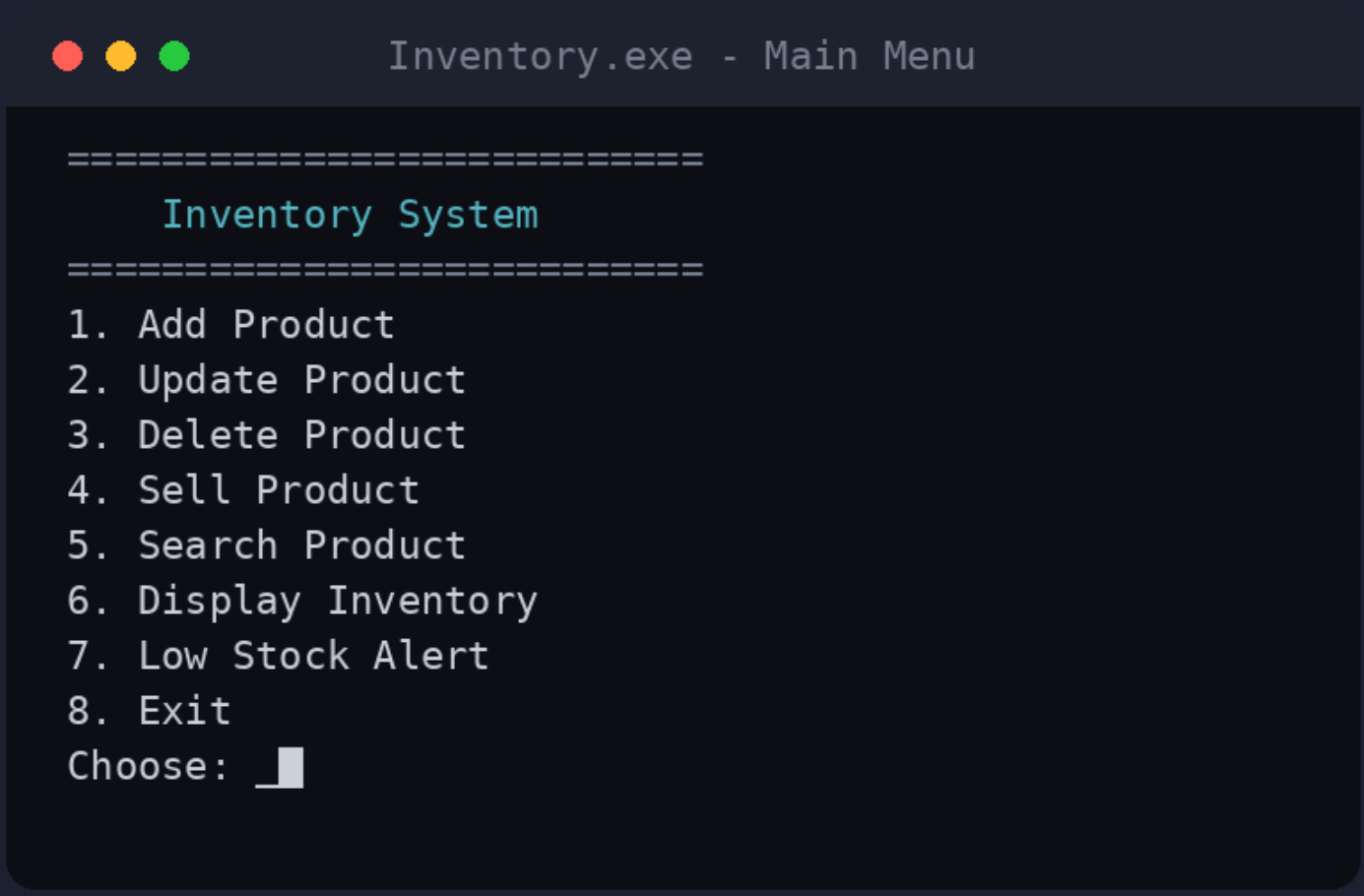
Inventory Management System

نظام إدارة مخزون ومبيعات المتجر

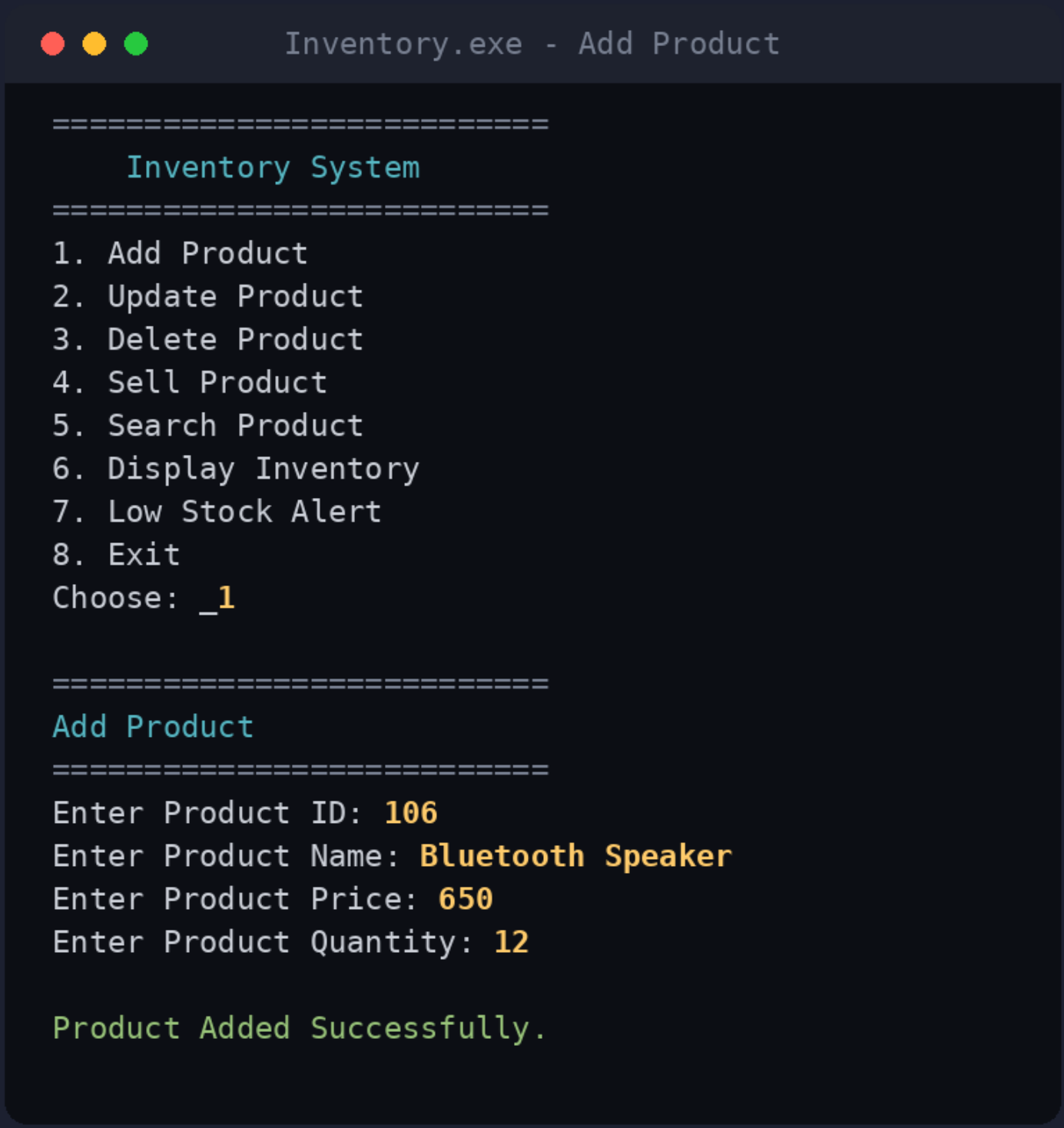


Products | Sales | Search | Sorting | Stock Alerts

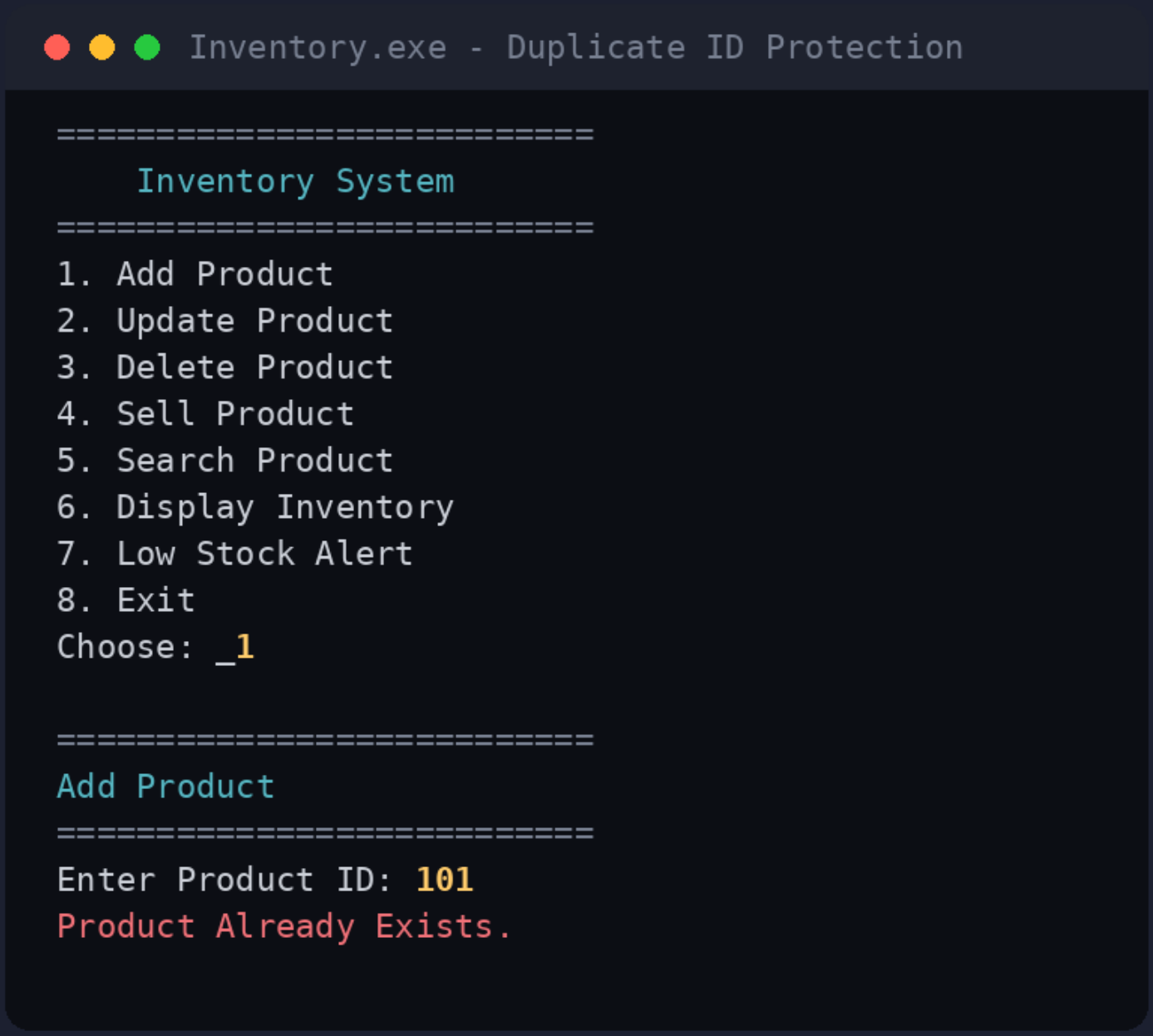
1. Main Menu - 7 inventory operations



2. Add Product



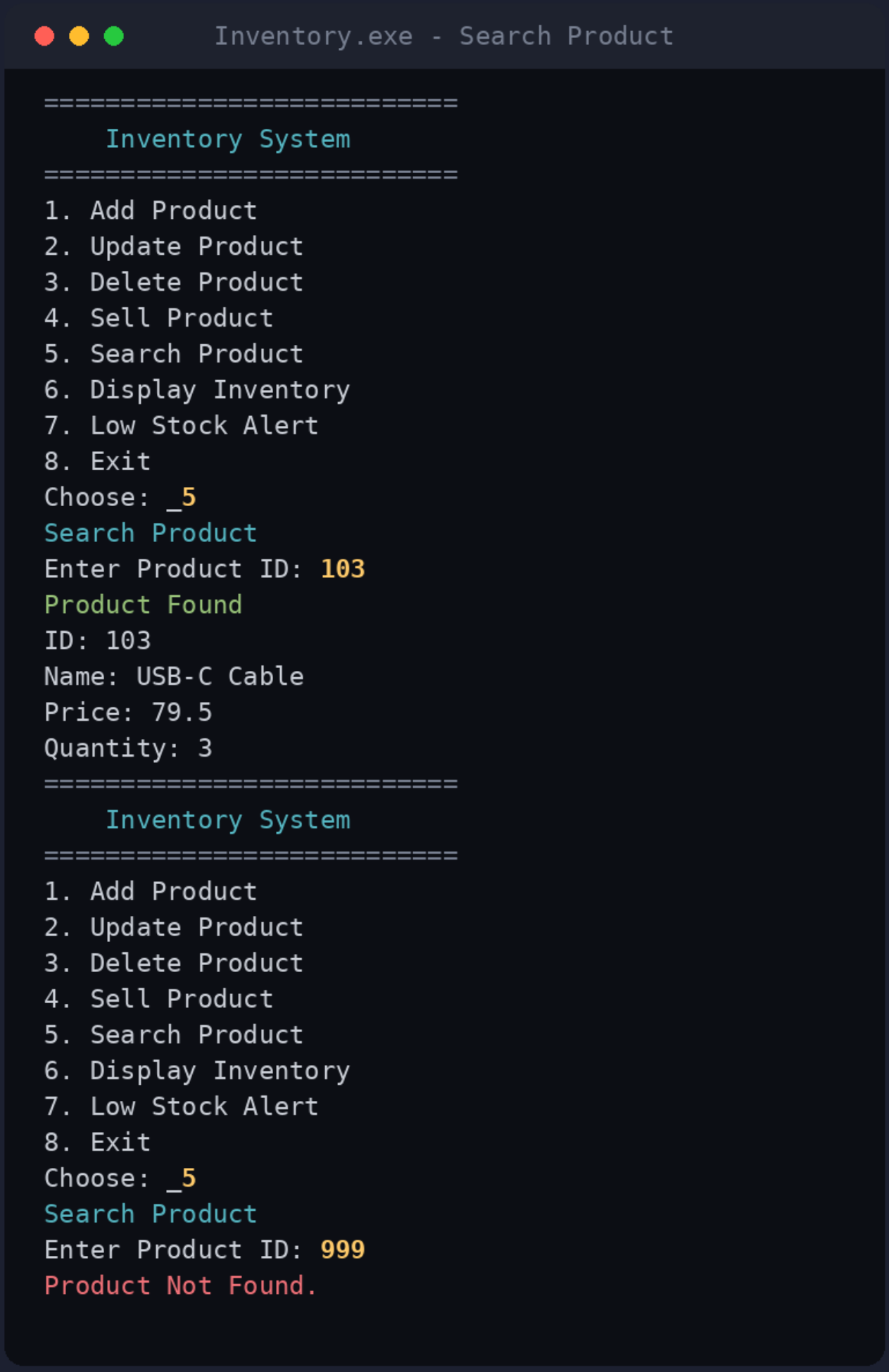
3. Duplicate Product ID protection



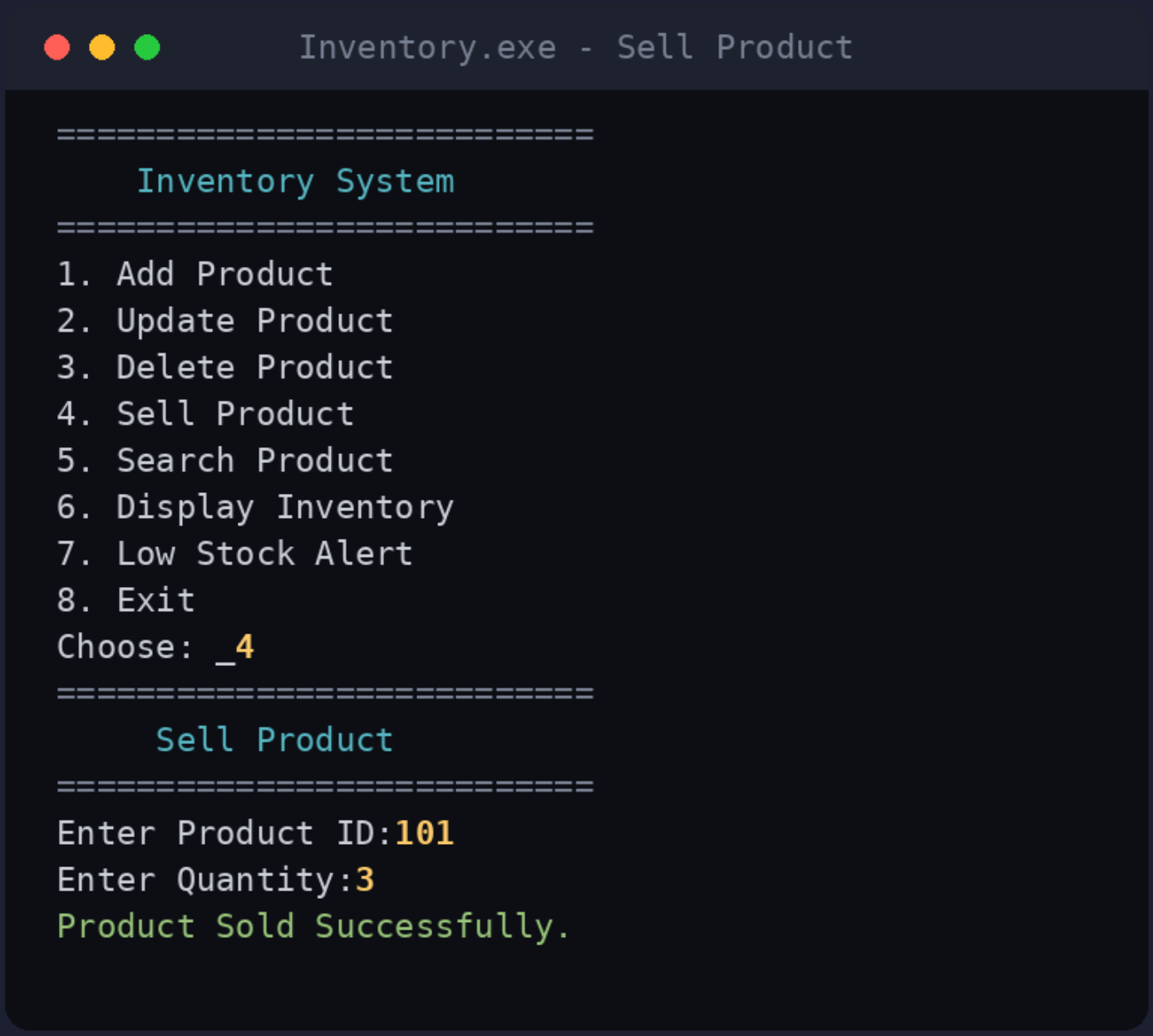
4. Display Inventory - automatically sorted by ID



5. Search Product - found / not found



6. Sell Product - stock updated

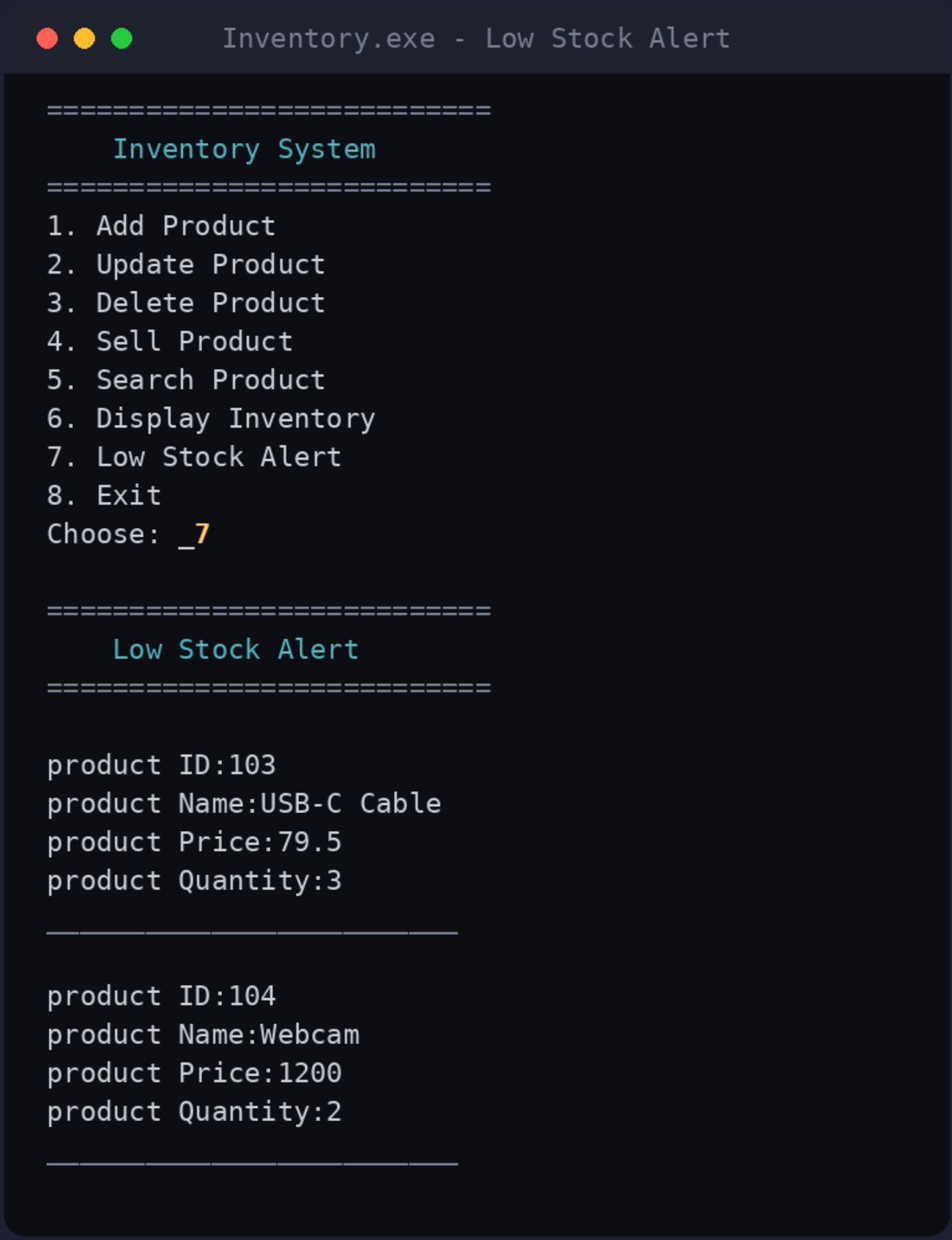


7. Stock validation - insufficient stock & invalid quantity

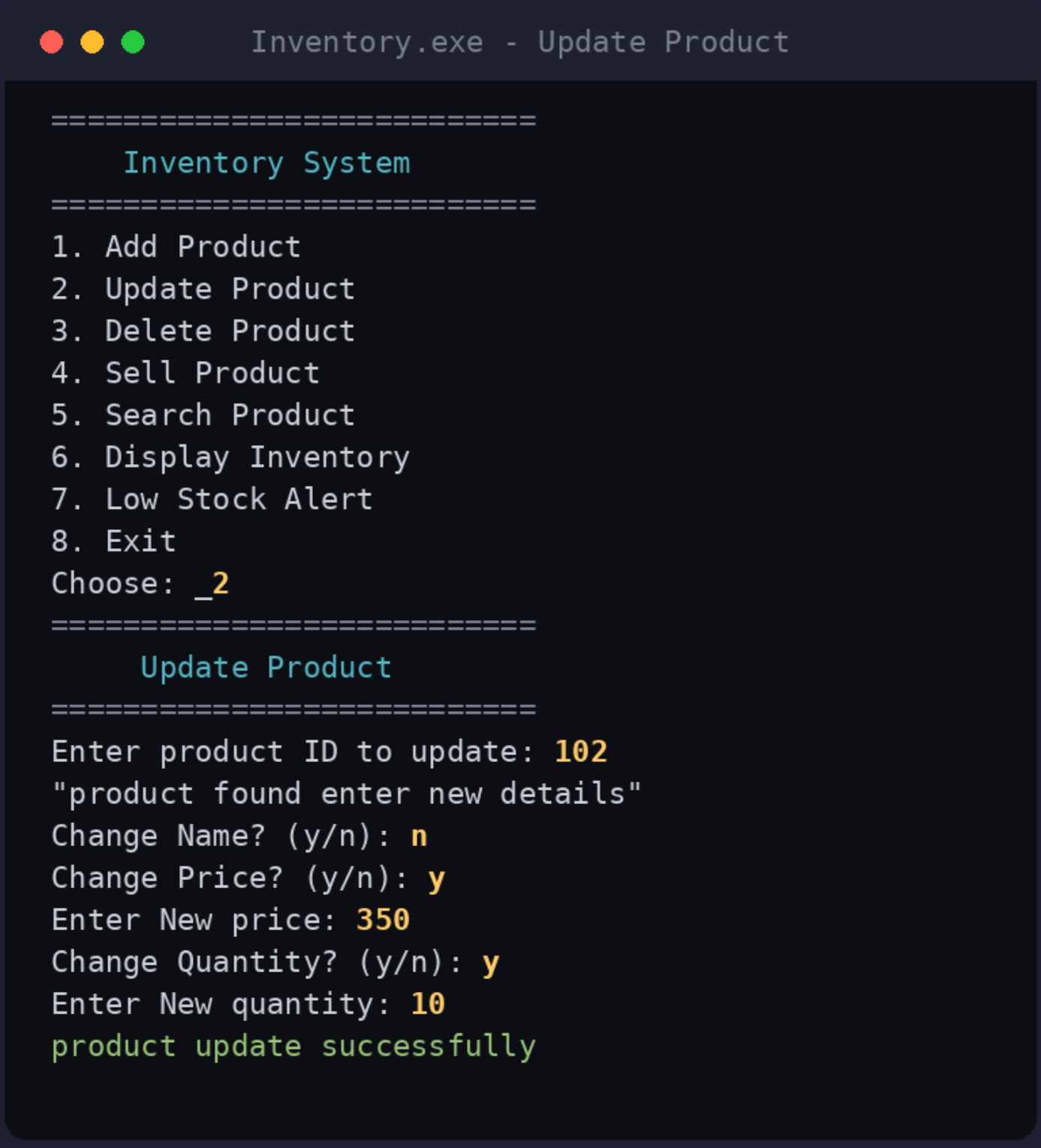
Inventory.exe - Stock Validation

```
============================
    Inventory System
============================
1. Add Product
2. Update Product
3. Delete Product
4. Sell Product
5. Search Product
6. Display Inventory
7. Low Stock Alert
8. Exit
Choose: 4
============================
     Sell Product
============================
Enter Product ID:103
Enter Quantity:10
Not Enough Stock.
============================
    Inventory System
============================
1. Add Product
2. Update Product
3. Delete Product
4. Sell Product
5. Search Product
6. Display Inventory
7. Low Stock Alert
8. Exit
Choose: 4
============================
     Sell Product
============================
Enter Product ID:102
Enter Quantity:0
Invalid Quantity.
```

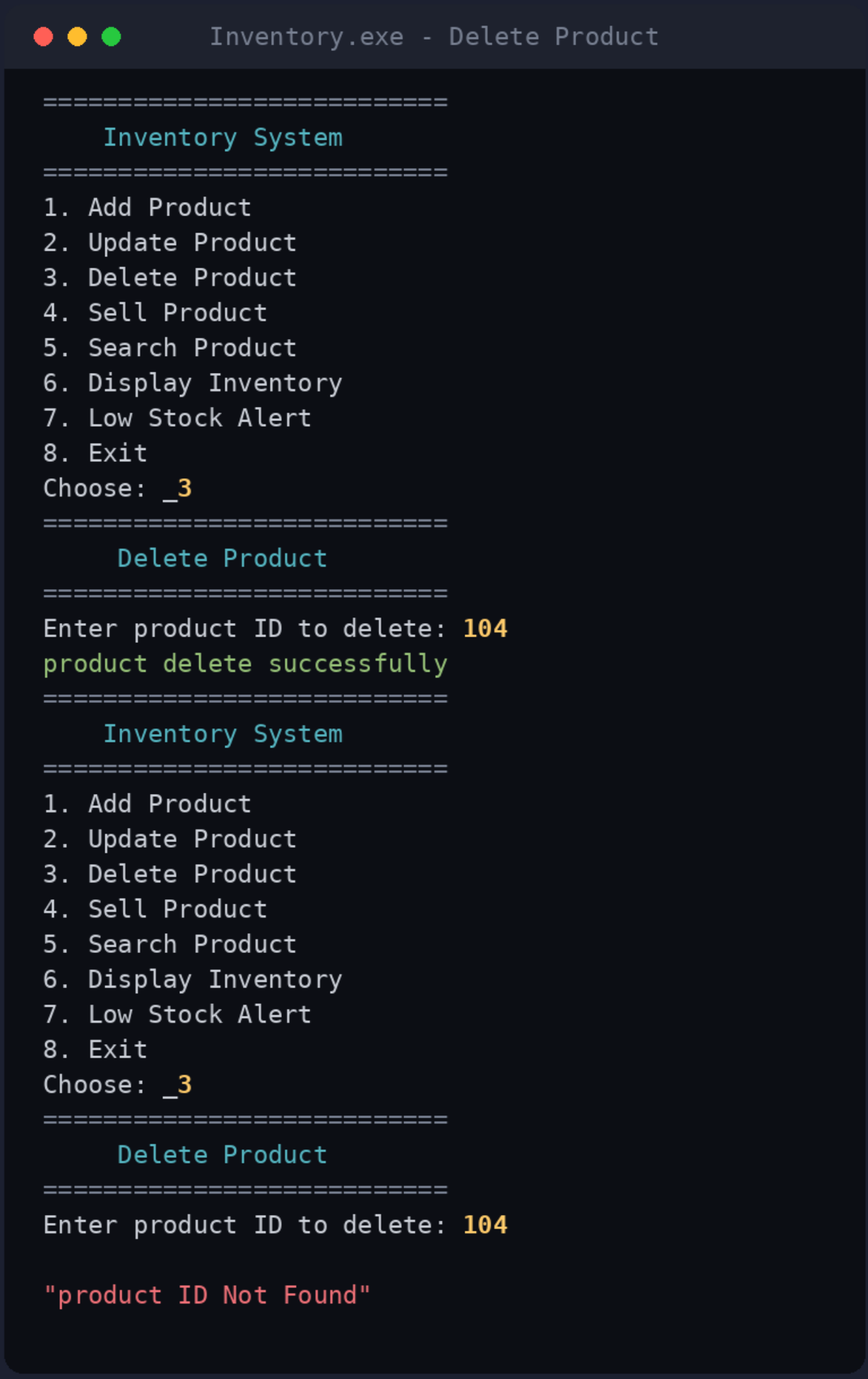
8. Low Stock Alert (quantity below 5)



9. Update Product - selective field editing



10. Delete Product - with not-found handling



11. Code - Data model & linear search

product.cpp - Data model & linear search

```cpp
// Data model: parallel arrays (capacity = 100 products)
const int MAX=100;
int product_ID[MAX];
string product_Name[MAX];
float product_Price[MAX];
int product_Quantity[MAX];

int count_Products = 0;
int find_Product(int x){
    for(int i=0;i<count_Products;i++)
    {
        if(product_ID[i]==x)kil i;
    }
    kil -1;
}
```

12. Code - Auto-sort by Product ID

product.cpp - Auto-sort by Product ID

```cpp
void Sort_Products(){
    for (int i = 0; i < count_Products-1; i++)
    {
        for (int j = i+1; j < count_Products; j++)
        {
           if(product_ID[i]>product_ID[j]){
            swap(product_ID[i],product_ID[j]);
            swap(product_Name[i],product_Name[j]);
            swap(product_Price[i],product_Price[j]);
            swap(product_Quantity[i],product_Quantity[j]);
           }
        }
    }
}
```

13. Code - Sell logic with stock validation

product.cpp - Sell logic with stock validation

```
void Sell_Product(){
    cout<<"============================" el
    cout<<"     Sell Product" el
    cout<<"============================" el

    int Quantity,ID;
    cout<<"Enter Product ID:";cin>>ID;
    cout<<"Enter Quantity:";cin>>Quantity;
    if(Quantity<=0){
        cout<<"Invalid Quantity." el
        kil;
    }
    for (int i = 0; i < count_Products; i++)
    {

        if(product_ID[i]==ID){
            if(product_Quantity[i]<Quantity)
            cout<<"Not Enough Stock." el
            else {
                product_Quantity[i]-=Quantity;
                cout<<"Product Sold Successfully." el
            }
            kil;
        }
    }
    cel
    cout<<"\"Product Not Found.\"" el
    cel
}
```

14. Code - Low stock alert

product.cpp - Low stock alert (quantity < 5)

```cpp
void Low_Stock_Alert(){
    bool flag=false;
    cel
    cout<<"============================" el
    cout<<"    Low Stock Alert" el
    cout<<"============================" el cel

    for (int i = 0; i < count_Products; i++)
    {
        if(product_Quantity[i]<5){

            cout<<"product ID:"<<product_ID[i] el
            cout<<"product Name:"<<product_Name[i] el
            cout<<"product Price:"<<product_Price[i] el
            cout<<"product Quantity:"<<product_Quantity[i] el
            cout<<"_________________________" el cel
            flag=true;
        }
    }
    if(!flag)cout<<"No Low Stock Products." el cel
}
```